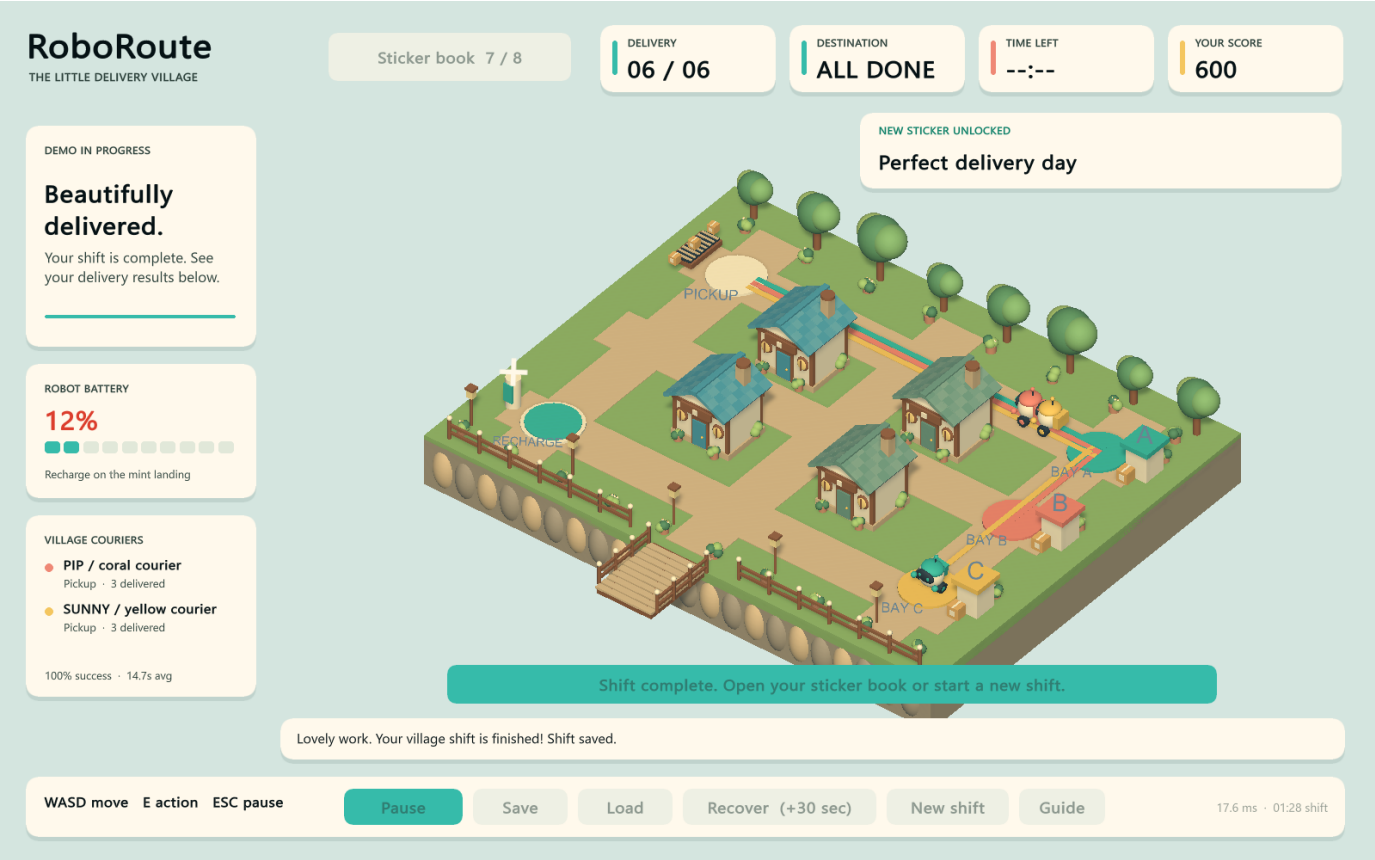
RoboRoute
Project Report

Cozy delivery simulation
Player experience and achievement edition

28 September 2026



A Unity engineering project with timed deliveries, autonomous couriers, persistent achievements and a guided player experience.

Development platform Unity 6000.3.10f1
Delivery platform Windows standalone
Authorship AI-assisted development

Executive Summary

RoboRoute turns a warehouse delivery simulation into a cozy isometric village. The player controls Sprout, a small courier robot, and delivers six parcels to correctly labelled bays. The project demonstrates C# state management, grid pathfinding, persistence, automated rule checks and player-facing interface design.

The current edition adds eight collectible achievements, a persistent sticker book, personal bests, a three-step guide, destination dots, context-sensitive action prompts, Escape-to-pause control and a ranked results screen. These additions make the existing delivery loop easier to understand and give players goals beyond finishing one shift.

The final build passed 118 automated assertions. A scripted standalone check delivered all six parcels, earned 600 points, missed zero jobs and recorded zero runtime exceptions or errors. Shift saves and career achievements both passed disk roundtrips. The driver was accelerated and programmatic, so these results do not substitute for manual keyboard acceptance testing or a controlled performance benchmark.

The project remains a single-village engineering prototype. It does not claim a full commercial campaign, multiplayer support or collision-free fleet scheduling. The report documents the implemented scope, its evidence and the remaining work.

Outcome	Evidence
Learnable controls	Three guide pages, contextual E action and a visible target marker
Replay goals	Eight named achievement rules, permanent unlocks and personal bests
Reliable delivery rules	118 passing assertions and a six-delivery standalone run
Reviewable delivery	Unity source package, Windows build, screenshots and this report

Table of Contents

Executive Summary 2

Table of Contents 3

1 Introduction 4

2 Requirements Analysis 5

3 System Design 6

3A Data Model and Persistence 7

4 Implementation 8

4A Player Interface 9

5 Testing and Evaluation 10

6 Conclusion and Future Work 11

7 References 12

Appendix A Easy Player Guide 13

Appendix B Achievement Reference 14

1 Introduction

A portfolio game needs both understandable interaction and explainable engineering. A delivery simulator offers visible evidence: a robot moves through an environment, collects a parcel, checks a destination, consumes battery and receives a result. A reviewer can connect each observed action to a small set of code responsibilities.

The original RoboRoute prototype already contained this delivery loop. Its industrial scene and dense interface made the experience less approachable. The cozy village revision introduced rounded couriers, timber cottages, colored routes and a warm interface. This edition extends that work with rewards and simpler guidance.

Objectives

Help a first-time player understand the next action without reading source code. Preserve the timing, battery, routing and persistence rules. Add deterministic achievements that cannot be repeatedly unlocked by loading the same save. Show meaningful results at the end of a shift and retain a personal record across shifts.

Scope

The implemented scope is one procedural 3D village, one controllable courier, two autonomous couriers, six timed jobs, three delivery bays, battery charging, recovery, JSON persistence, eight achievements and a reusable guide. The environment is generated at runtime from original geometry. Referenced illustrations informed its visual direction rather than being used as a flat game background.

Report approach

The report follows the requested introduction, requirements, design, implementation, testing and conclusion structure. Requirements are mapped to observable behavior and tests, consistent with the communication and traceability emphasis of the FHSU project-report guidance [1].

2 Requirements Analysis

The primary player wants to know where to go, which button to press, why an action failed and what was achieved. The reviewer wants to inspect the implementation and distinguish tested behavior from future plans.

ID	User story and requirement	Acceptance condition
FR01	As a player, I can move and interact.	WASD/arrows move relative to the camera; E or the nearby action button interacts.
FR02	I can deliver to the correct bay.	A valid delivery gives 100 points once; a wrong bay retains the parcel.
FR03	I can understand the deadline.	Each dispatched job has 75 seconds; six resolved jobs end the shift.
FR04	I can manage battery safely.	Travel consumes energy; the dock charges; recovery adds 30 seconds.
FR05	I can collect achievements.	Eight deterministic rules unlock permanent stickers without duplicate rewards.
FR06	I can learn without guessing.	Three guide pages, glowing target dots, bay letters and contextual prompts are visible.
FR07	I can save and return.	Validated shift and career JSON load; old version-2 shift saves remain readable.
FR08	I can replay and compare results.	New shifts retain stickers and personal bests; results show stars and streaks.

System requirements and constraints

The project uses Unity 6000.3.10f1, C#, the Universal Render Pipeline and Input System. The verified target is a Windows desktop with a keyboard, mouse and Direct3D-capable graphics. The inspected game layout is 1600 by 1000 pixels. Minimum hardware specifications and other platforms have not been established by benchmarking. No network service or database server is required to play.

3 System Design

The design separates plain data and deterministic rules from input, world generation and presentation. ShiftController coordinates the game loop, while ShiftState remains serializable and testable. AchievementRules evaluates shift facts; CareerProfile retains monotonic unlocks and personal bests.

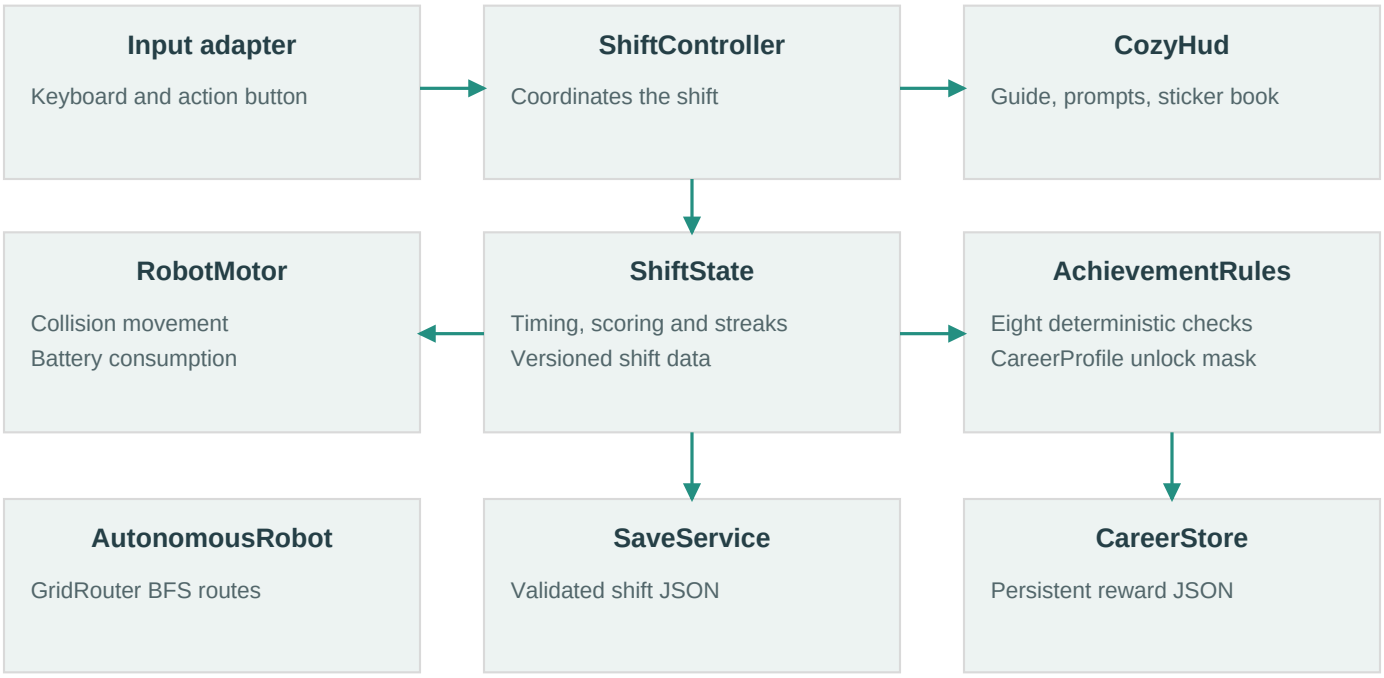


Figure 1 Architecture and primary data dependencies

Delivery data flow

Input requests an interaction. ShiftController checks the player position and delegates to ShiftState. A matching bay changes cargo, score, streak and timing totals. AchievementRules evaluates the resulting state, CareerProfile records new unlocks, the HUD queues feedback, and SaveService writes the shift. A wrong bay changes only the attempt counter and feedback; cargo and score remain intact.

Autonomous navigation

Each courier follows an explicit pickup, delivery or charging state. GridRouter uses a FIFO frontier and predecessor map to search equal-cost neighboring cells. Rack bounds are inflated for clearance, and village buildings keep the same obstacle footprints. Couriers yield to the player; robot-to-robot traffic reservations remain future work.

3A Data Model and Persistence

RoboRoute uses two local JSON documents rather than a relational database. The shift document stores resumable game facts; the career document stores permanent unlocks and best results. Test runs use separate filenames to protect ordinary player progress.

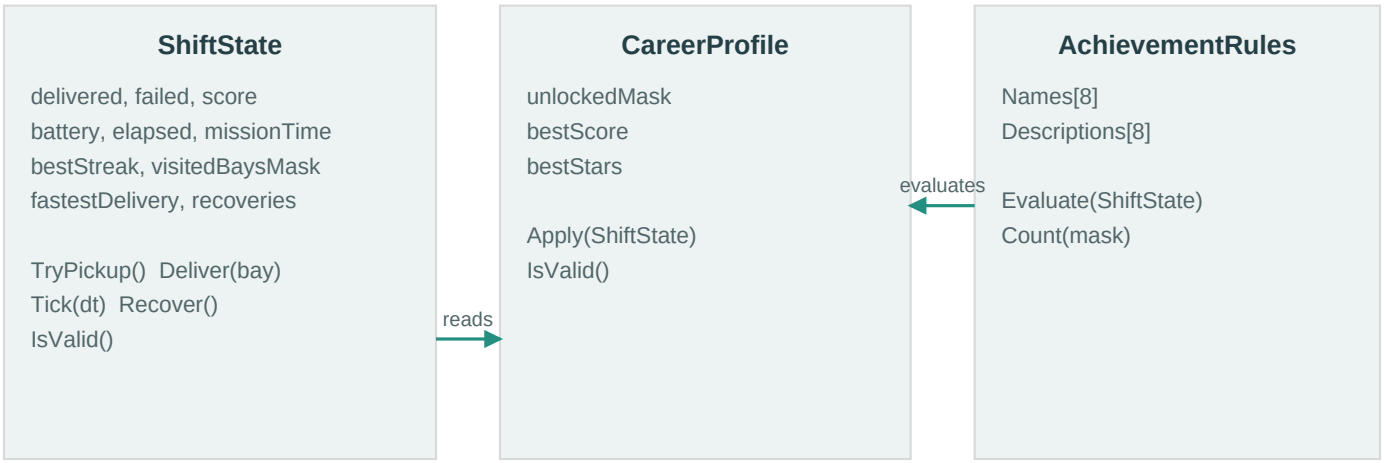


Figure 2 Core class responsibilities and relationships

Document	Fields and validation
Shift version 2	Delivered, failed, score, cargo, battery, timers, streaks, wrong-bay attempts, recoveries, charge count, visited bays and fastest delivery. Reject non-finite time or battery values and impossible totals.
Career version 1	Eight-bit unlockedMask, bestScore and bestStars. Reject unknown bits, scores outside 0 to 600, nonmultiples of 100 and ranks outside 0 to 3.
Write and restore	Serialize to a temporary file, then replace the destination while retaining a backup. Load only validated data. Shift load places the player at the charging dock.

New shift fields are optional in existing version-2 JSON. Missing achievement facts default to zero, so old saves remain readable without inventing past streaks or charging visits. Career unlocks are combined with bitwise OR, making repeated application idempotent. There is no cloud synchronization, encryption or complete fleet-state persistence.

4 Implementation

Tools and code organization

Unity supplies rendering, object lifecycle, input and CharacterController movement. RobotMotor converts keyboard input into camera-relative movement and applies battery consumption to actual horizontal travel. CharacterController.Move applies a movement delta subject to collisions [2]. Unity JsonUtility serializes the plain state classes used by the save services [3].

Module	Responsibility
ShiftState	Pickup capacity, deadline progression, validated delivery, streaks, rank and state invariants
AchievementRules and CareerProfile	Eight badge predicates, bitmask union and personal bests
CareerStore and SaveService	Validated JSON disk writes and reads
CozyHud and ShiftController	Guide pages, sticker collection, queued toasts, action prompts and results
ObjectiveBeacon	Visible next destination, with low-battery dock priority
WarehouseBuilder and CozyMotion	Original village geometry, robot faces, conveyors, charging pulse and delivery effects
BuildProject and AchievementChecks	Repeatable build and domain assertions

Reward and timing rules

Accepted deliveries earn a fixed 100 points. Rank is based on delivered parcels: one star for 1 to 3, two for 4 to 5, and three for all 6. Sticker rewards do not alter scoring or battery. The fastest-delivery threshold is inclusive at 20 seconds; the last-second badge starts at 65 seconds. A timeout clears the active streak but preserves the best streak.

The total delivery duration is capped at the authoritative elapsed clock when a delivery completes. This prevents floating-point accumulation from making a valid shift unsaveable. The correction preserves the score and JSON schema.

4A Player Interface

The interface prioritizes the current action. Bay letters supplement route colors. The header keeps delivery number, destination, time and score visible; the left column shows the parcel objective, battery and autonomous couriers. A nearby action prompt supports both keyboard interaction and a clickable alternative.

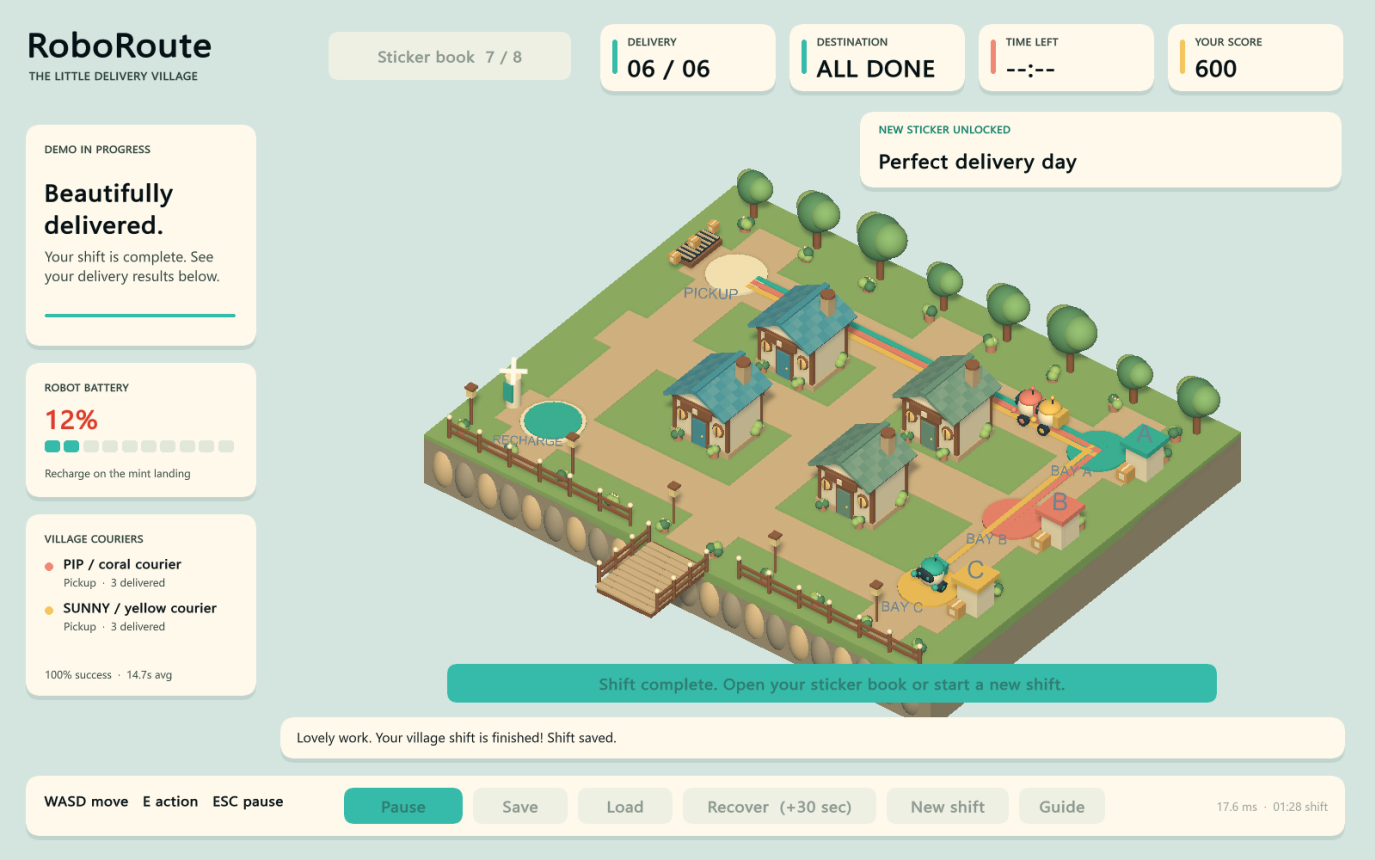


Figure 3 Verified game HUD after the scripted shift

Guide and feedback

The first-time guide is divided into three short pages: movement, the pickup-and-delivery loop, and charging with achievements. Guide and sticker-book panels pause the simulation and disable controls behind them. The glow marker moves to the charger below 25 percent battery. New unlocks are queued as brief notifications rather than interrupting movement.

Fun and clarity

Parcel names such as a robot birthday cake and a very small hat add character without changing rules. Results praise progress and suggest a next action. Failed attempts identify the correct bay instead of only displaying an error. Humor remains secondary to clear instructions.

5 Testing and Evaluation

The final Unity build completed successfully. The automated domain runner passed 118 assertions. The standalone integration run used the real scene, grid routes, movement speed, battery rules and file services, while driving the player programmatically at an accelerated time scale.

Test layer	Verified result
Domain assertions	118 passed: scoring, wrong bay, duplicate delivery, timeouts, path clearance, JSON rules and achievement boundaries.
Compatibility and rewards	Existing version-2 saves remain valid; unknown career bits are rejected; repeated achievement application returns no duplicate unlock.
Standalone shift	6 delivered, 0 missed, 600 points, 0 runtime errors or exceptions; process exited with code 0.
Disk persistence	Shift save/load roundtrip and career unlock/personal-best roundtrip passed using isolated test files.
Visual checks	HUD, guide, sticker book and results screenshots inspected at 1600 by 1000.
Manual acceptance	Keyboard behavior, subjective audio and broad resolution/accessibility testing still require direct user acceptance.

Bugs found and resolved

Earlier runtime checking exposed a shader stripped from the standalone build; an explicit Resources material corrected it. Repeat runs exposed a floating-point timing invariant that could reject a valid save; the delivery-time cap and regression checks address it. The new guide and collection panels explicitly disable controls behind them. Restart now resets autonomous couriers while retaining the career profile.

Performance interpretation

The scene-check snapshot contains roughly one thousand renderers. The HUD frame-time indicator is observational only. Capture overhead, accelerated simulation and a capped frame rate prevent this test from establishing a performance guarantee. Profiling allocations, draw calls and worst-case frame times is planned before expanding the map.

6 Conclusion and Future Work

RoboRoute now offers a complete short shift with clear instructions, persistent achievement goals and a results screen. Its strongest engineering evidence is the link between visible behavior, small rule classes and repeatable tests. The player can understand the next step, recover from mistakes and carry a collection into another shift.

Known limitations

The project is a single-village prototype with a fixed six-job schedule. The autonomous fleet has no reservation system for robot-to-robot traffic. Saves restore the player shift at the dock rather than reproducing every world position. Input acceptance, subjective sound quality, alternative screen sizes and minimum hardware have not been certified by this automated session.

Prioritized next work

Priority	Improvement	Completion evidence
1	Manual acceptance and accessibility	A completed keyboard, mouse, audio and multi-resolution checklist
2	Fleet traffic reservations	No mutual blocking in repeatable crossing scenarios
3	Configurable mission data and difficulty	Validated mission assets and difficulty-specific tests
4	Performance and asset optimization	Measured CPU/GPU timings and allocation budgets
5	More locations and cosmetic rewards	New content that preserves tested rules and save compatibility

Portfolio presentation

Explain the implementation as an AI-assisted project and be ready to discuss the decisions: why a bitmask makes rewards idempotent, why BFS works for equal-cost cells, why schema validation matters, and why automated movement tests do not prove keyboard usability. Publication to a public repository is a separate action.

7 References

[1] Fort Hays State University

Software Engineering Project Report Requirements. Accessed 11 September 2026.

https://www.fhsu.edu/cs-informatics/csci441/report1

Used for report communication and requirement traceability guidance.

[2] Unity Technologies

CharacterController Move API. Accessed 11 September 2026.

https://docs.unity3d.com/6000.3/Documentation/ScriptReference/CharacterController.Move.html

Supports the movement and collision API description.

[3] Unity Technologies

JsonUtility API. Accessed 11 September 2026.

https://docs.unity3d.com/6000.3/Documentation/ScriptReference/JsonUtility.html

Supports the JSON serialization API description.

Project evidence

Assets/RoboRoute/Runtime and Assets/RoboRoute/Editor contain the implementation and assertion runners. Evidence/domain-checks.txt records the latest assertion count. Evidence/achievements-integration-check.txt records the final standalone result. These are local project records, not independent user research.

Visual references

The supplied cozy village and robot illustrations informed the art direction. The game uses procedural geometry and code-generated presentation. No external illustration is used as an interactive background. This report does not claim third-party asset ownership or fabricated team contributions.

Appendix A Easy Player Guide

1 Start your delivery day

Read the three short guide pages, then choose Let's deliver. You control the mint-colored robot, Sprout.

2 Find your parcel

Use WASD or the arrow keys. Follow the glowing dots to PICKUP. Press E or click the nearby action button.

3 Match the bay letter

Read the destination at the top. Go to BAY A, B or C, then press E. Matching the letter matters; a wrong bay keeps your parcel.

4 Keep your battery happy

If battery is low, follow the dots to RECHARGE and wait. Charging is automatic. Recover returns you to the dock but adds 30 seconds.

5 Finish and collect

Resolve six jobs to finish a shift. See your score and stars, then open the sticker book. Your collection and personal bests stay for the next shift.

Control	Action
WASD or arrows	Move relative to the camera
E or nearby action button	Pick up or deliver
ESC or Pause	Pause or resume
Guide	Read the instructions again
Sticker book	See collected and locked achievements
Save and Load	Store or restore the player shift

Remember: the 75-second timer begins when a job is dispatched, not when its parcel is picked up. You carry one parcel at a time. No need to rush blindly: match the bay letter and recharge early.

Appendix B Achievement Reference

Collected stickers stay unlocked across new shifts and repeated loads. A best score and best star rating are retained separately from current shift progress. Achievements are cosmetic goals and do not add score or change movement.

Sticker	How to earn it
First class parcel	Complete one delivery.
Three in a row	Deliver three jobs in a row without a timeout.
Speedy sprout	Complete one job within 20 seconds of dispatch.
Powered by plants	Visit the dock below 80% battery and charge to at least 95%.
Village explorer	Deliver to all three bays in one shift.
Perfect delivery day	Complete a shift with all six parcels delivered.
Careful courier	Deliver all six with no wrong-bay attempt and no recovery.
Just in thyme	Deliver during the last ten seconds of a job, from 65 seconds onward.

A simple challenge ladder

First, earn First class parcel. Next, aim for Three in a row and visit every bay. Once the route feels familiar, try Perfect delivery day. Finish by collecting the harder speed, care and last-second stickers. The game never requires an achievement to continue playing.

Test execution appendix

Run the installed Unity editor in batch mode with the project path and executeMethod RoboRoute.Editor.BuildProject.Check to run domain assertions. BuildProject.Windows creates the Windows build. The standalone -roboroute-demo -fast-smoke arguments run an isolated scripted shift and save evidence under Build/Evidence/CozyRun. The automated driver is a test facility, not a normal play mode.